

DESENVOLVIMENTO DE UM SIMULADOR BLINDADO DE BATALHA NAVAL MULTIPLAYER NA LINGUAGEM C

Bruno Cesar Perillo da Silva

bruno.silva19@uscsonline.com.br

Gustavo Gonçalves Paulo de Almeida

gustavo.almeida@uscsonline.com.br

Luiggi Gouveia Bincoletto

luiggi.bincoletto@uscsonline.com.br

Orientador: Prof. Me. Fábio Ferreira de Assis

fabio.assis@online.uscs.edu.br

RESUMO

Este projeto apresenta o desenvolvimento de um simulador tático de Batalha Naval multiplayer local, implementado inteiramente na linguagem de programação C para consoles de comando. O objetivo principal do trabalho foi construir um software com arquitetura robusta e blindada contra falhas críticas de input geradas pelos usuários. A metodologia consistiu na aplicação prática de conceitos de programação estruturada, loops aninhados, matrizes bidimensionais e manipulação direta de buffers de memória de entrada (stdin). Como resultados obtidos, o sistema demonstrou imunidade total a travamentos por estouro de inteiros (integer overflow), tratou de forma automatizada tentativas de trapaça por sobreposição de embarcações e garantiu a persistência visual do resultado em arquivo executável autônomo, validando a integridade do fluxo do jogo em ambiente de testes contínuos.

Palavras-chave: Batalha Naval. Linguagem C. Tratamento de Erros. Buffer de Teclado. Engenharia de Software.

1. INTRODUÇÃO

O desenvolvimento de jogos de tabuleiro digitais via console de comandos atua como um excelente mecanismo para a consolidação prática de fundamentos de algoritmos e lógica de programação estruturada. Este projeto descreve o ciclo de vida e a construção de um jogo de Batalha Naval para dois jogadores simultâneos operando em modo de turnos alternados.

A linguagem de programação C foi selecionada como base de desenvolvimento devido ao seu controle de baixo nível sobre a memória, gerenciamento direto de fluxos de entrada/saída (I/O) e portabilidade de compilação. Ao projetar o simulador, os principais conceitos acadêmicos explorados englobaram a declaração e varredura de matrizes bidimensionais (para a representação gráfica do mapa oceânico), estruturas condicionais aninhadas (para validação de fronteiras de ataque) e laços de repetição interativos (para o controle cíclico dos turnos).

A relevância do trabalho fundamenta-se na simulação de cenários reais de falhas humanas no console de entrada. O projeto aplica técnicas de Engenharia de Software para mitigar vulnerabilidades comuns em programas de terminal de comandos, focando no desenvolvimento de rotinas para o tratamento de exceções de digitação que comumente causam travamentos indesejados.

2. DESENVOLVIMENTO

O sistema foi projetado seguindo o paradigma estruturado da linguagem C, utilizando ambientes de compilação compatíveis com o padrão GCC (*GNU Compiler Collection*). A arquitetura do software inicia-se com a diretiva de inclusão de duas bibliotecas essenciais para a operação do sistema via console: a `<stdio.h>` (*Standard Input/Output*), responsável por gerenciar os fluxos fundamentais de entrada e saída de dados (como as funções `printf` e `scanf`), e a `<stdlib.h>` (*Standard Library*), responsável por fornecer funções utilitárias de gerenciamento de sistema (como a função `system`, indispensável para os comandos de limpeza e retenção de tela).

A partir dessas diretivas base, foram aplicadas técnicas de varredura matricial iterativa, controle de fluxo por chaveamento de variáveis sinalizadoras (*flags*) e gerenciamento mecânico do buffer padrão de entrada de dados (*stdin*).

Como modelo de inspiração para este sistema, o grupo tomou por base a arquitetura de matrizes estáticas amplamente discutida nos manuais de estruturas de dados e jogos por console simplificados (como campo minado e jogos de tabuleiro). Adaptou-se a matriz clássica de caracteres para guardar estados dinâmicos como água (.), tiro errado (~) e navio abatido (X).

2.1. Regras do Jogo

A Batalha Naval tradicional baseia-se nas seguintes diretrizes gerais de mecânica de jogo:

- **Jogadores:** O jogo é composto por dois competidores que atuam de forma alternada (sistema de turnos).
- **Posicionamento:** Cada jogador possui uma frota de embarcações disposta ocultamente em seu respectivo quadrante oceânico.
- **Ataque:** Na sua vez, o jogador informa um par de coordenadas (linha e coluna) no mapa inimigo.
- **Vitória:** O jogo termina e declara o vencedor no instante exato em que um dos jogadores conseguir localizar e abater todos os alvos da frota adversária primeiro.

2.2. Regras Aplicadas no Projeto

O simulador reduziu a escala do mapa tradicional para matrizes quadradas de tamanho 5×5 com índices de 0 a 4, fixando a frota de cada competidor em exatamente 3 navios de 1 bloco. As regras efetivamente implementadas via código foram:

- **Ocultamento e Sigilo:** Chamadas de sistema para limpeza de console (`system("cls")`) garantem que um competidor não descubra onde o oponente escondeu seus barcos.
- **Alternância de Turnos Automática:** O sistema gerencia a troca de vez entre os competidores através do encerramento de escopo dos blocos de ataque.
- **Deteção de Estado de Célula:** Validação ativa para impedir que um competidor pontue de forma duplicada ao atirar em uma célula com navio já destruído (X).
- **Punição Estratégica por Repetição:** Caso o usuário insira coordenadas já atacadas anteriormente (seja água ou navio abatido), o turno prossegue normalmente, penalizando o jogador desatento com a perda da vez.
- **Condição Terminal Limiar:** Verificação matemática contínua do score. Assim que um jogador atinge a meta de 3 acertos, o laço de combate encerra instantaneamente.

2.3. Documentação do Sistema (Imagens e Código)

Nesta seção estão evidenciados todos os módulos e blocos de refatoração desenvolvidos pelo grupo para garantir a consistência de memória, segurança de dados e a estabilidade de fluxo do software:

Módulo 1: Loop de Validação de Fronteiras do Mapa (Conserto #1)

Este módulo implementa uma das principais estratégias de controle de fluxo do sistema, combinando de forma aninhada as estruturas ‘for’ e ‘while(1)’. O laço externo ‘for’ atua de maneira preditiva, gerenciando a meta quantitativa do programa (coletar a posição de exatamente 3 navios). Logo abaixo dele, o laço estruturado ‘while(1)’ cria uma barreira de segurança contínua que atua diretamente contra a previsibilidade de erros ou a insistência do usuário em fornecer dados inválidos. Se o operador inserir coordenadas fora dos limites geográficos do tabuleiro (5 x 5, índices de 0 a 4) ou posições já ocupadas pelo histórico de jogadas, o fluxo permanece retido por tempo indeterminado na mesma rodada do cadastro. O programa só concede permissão para avançar ao próximo índice incremental do ‘for’ quando uma entrada legítima satisfizer todas as condições da lógica de proteção; nesse instante, o comando ‘break’ é disparado, rompendo o isolamento do loop infinito.

```
150     for(i = 0; i < 3; i++) {
151         while(1) {
152             printf("\nNAVIO %d\n", i + 1);
153             printf("Linha (0-4): ");
154             if (scanf("%d", &navio2Linha[i]) != 1) {
155                 navio2Linha[i] = -1;
156                 while(getchar() != '\n');
157             }
158
159             printf("Coluna (0-4): ");
160             if (scanf("%d", &navio2Coluna[i]) != 1) {
161                 navio2Coluna[i] = -1;
162                 while(getchar() != '\n');
163             }
164
165             if(navio2Linha[i] >= 0 && navio2Linha[i] <= 4 &&
166                navio2Coluna[i] >= 0 && navio2Coluna[i] <= 4) {
167
168                 posicaoOcupada = 0;
169                 for(k = 0; k < i; k++) {
170                     if(navio2Linha[i] == navio2Linha[k] &&
171                        navio2Coluna[i] == navio2Coluna[k]) {
172                         posicaoOcupada = 1;
173                         break;
174                     }
175                 }
176
177                 if(posicaoOcupada == 1) {
178                     printf("\n[ERRO] Voce ja colocou um navio nesta posicao! Escolha outra.\n");
179                 } else {
180                     break;
181                 }
182             } else {
183                 printf("\n[ERRO] Posicao invalida! O mapa tatico so vai de 0 a 4.\n");
184                 while(getchar() != '\n');
185             }
186         }
```

Módulo 2: Estabilização de Interface e Retenção de Fluxo (Conserto #2)

Implementação do comando nativo `system("pause")` para eliminar o bug do "efeito fantasma" causado por resíduos do caractere `\n` no teclado, impedindo que o terminal limpe os alertas antes da leitura.

```
132 printf("\nNavios posicionados com sucesso!\n");
133
134 /* =====
135  CONCERTO #2: ADOÇÃO DO 'system("pause")' DO WINDOWS
136  - Por que arrumamos: Usar 'getchar()' ou 'fflush(stdin)' gerava o bug do
137  "efeito fantasma". O enter oculto pulava a leitura e o 'system("cls")'
138  limpava a tela sem dar tempo para os jogadores lerem as mensagens de erro ou acerto.
139  =====
140 */
141 system("pause");
142 system("cls");
```

Módulo 3: Intercepção de Inputs Corrompidos por Texto (Conserto #3 Partes A e B).

Este módulo detalha a estratégia de engenharia adotada para neutralizar travamentos por injeção de caracteres textuais ou estouro de inteiros (*integer overflow*). Visto que a falha de entrada no terminal compromete o fluxo em diferentes etapas do sistema, a mesma lógica defensiva baseada na validação do retorno da função `scanf` foi dividida e aplicada em duas fases distintas do simulador:

- **Parte A (Fase de Cadastro):** Evita que o programa entre em pane ou loops infinitos de interface caso o usuário digite caracteres inválidos no momento de configurar as coordenadas dos seus navios ocultos.
- **Parte B (Fase de Combate):** Protege a integridade da rodada tática contra tentativas de manipulação ou "ataques troll" durante os disparos de bombas, forçando o sistema a converter o erro em um tiro perdido fora do mapa sem interromper a execução do binário.

Se o retorno da leitura for diferente de 1 (indicando que o dado inserido não é um inteiro válido), o algoritmo força a respectiva variável de controle a assumir o valor de erro -1 e aciona um triturador mecânico de buffer (`while(getchar() != '\n')`), limpando toda a fila de memória residual até a quebra de linha.

Código Fonte — Parte A (Cadastro):

```
80
81 /* =====
82  CONCERTO #3 (PARTE A): TESTE DE RETORNO DO SCANF NO CADASTRO
83  - Por que arrumamos: Se o usuário digitasse letras ou strings contínuas
84  (como '-7-7-7-7'), o scanf falhava (retorno != 1), deixando lixo no buffer.
85  Nós capturamos a falha, forçamos o valor '-1' (inválido) e limpamos a memória.
86  =====
87  */
88 if (scanf("%d", &navio1Linha[i]) != 1) {
89     navio1Linha[i] = -1;
90     while(getchar() != '\n'); // Triturador de lixo do buffer
91 }
```

Código Fonte — Parte B (Combate):

```
215  /* =====  
216  CONCERTO #3 (PARTE B): BLINDAGEM DE COMBATE CONTRA ATAQUE TROLL (-7-7-7-7)  
217  - Por que arrumamos: Se o usuário injetar strings massivas ou gerar um  
218  Integer Overflow (estouro de inteiro), o scanf falha retornando 0.  
219  Nós forçamos a linha a valer '-1' e limpamos o buffer imediatamente.  
220  O jogo avisa que a coordenada é inválida e não entra em loop infinito.  
221  =====  
222  */  
223  printf("\nDigite a linha do ataque: ");  
224  if (scanf("%d", &linha) != 1) {  
225      linha = -1;  
226      while(getchar() != '\n'); // Limpa tudo até achar a quebra de linha (ENTER)  
227  }
```

Módulo 4: Varredura de Histórico Contra Sobreposição de Navios (Concerto #4)

Este módulo utiliza um laço retrospectivo `for(k = 0; k < i; k++)` controlado por uma variável sinalizadora (`flag posicaoOcupada`) para impedir a trapaça ou erro de colocar mais de um navio no mesmo quadrante.

```
102  /* =====  
103  CONCERTO #4: RASTREIO HISTÓRICO CONTRA SOBREPOSIÇÃO DE NAVIOS  
104  - Por que arrumamos: Se um jogador colocasse mais de um barco na mesma casa,  
105  o mapa ficaria com menos alvos do que o necessário para vencer. Isso tornava  
106  matematicamente impossível atingir 3 acertos, travando o jogo no combate.  
107  O laço 'k' varre o passado e impede a duplicidade.  
108  =====  
109  */  
110  posicaoOcupada = 0;  
111  for(k = 0; k < i; k++) {  
112      if(navio1Linha[i] == navio1Linha[k] &&  
113          navio1Coluna[i] == navio1Coluna[k]) {  
114          posicaoOcupada = 1;  
115          break;  
116      }  
117  }
```

Módulo 5: Limitação de Caracteres nas Strings de Nome (Conserto #5)

Substituição da máscara padrão por `scanf("%29s")` para estabelecer um teto rígido de armazenamento, blindando o programa contra ataques de String Buffer Overflow que corrompiam o console.

```
43  /* =====
44  CONserto #5: LIMITAÇÃO DE CARACTERES NAS STRINGS (MÁSCARA %29s)
45  - Por que arrumamos: O 'scanf("%s")' antigo aceitava strings infinitas.
46  | Se o usuário digitasse nomes gigantes, ocorria um Buffer Overflow (estouro
47  | de buffer), corrompendo as variáveis vizinhas na memória. Além disso,
48  | limpamos o buffer com 'while(getchar())' caso o usuário digite além da conta.
49  =====
50  */
51  printf("\nNome do Jogador 1 (Max 29 letras): ");
52  scanf("%29s", nome1);
53  while(getchar() != '\n');
54
55  printf("Nome do Jogador 2 (Max 29 letras): ");
56  scanf("%29s", nome2);
57  while(getchar() != '\n');
```

Módulo 6: Interrupção de Laço e Passagem Alternada de Turno (Módulo do break)

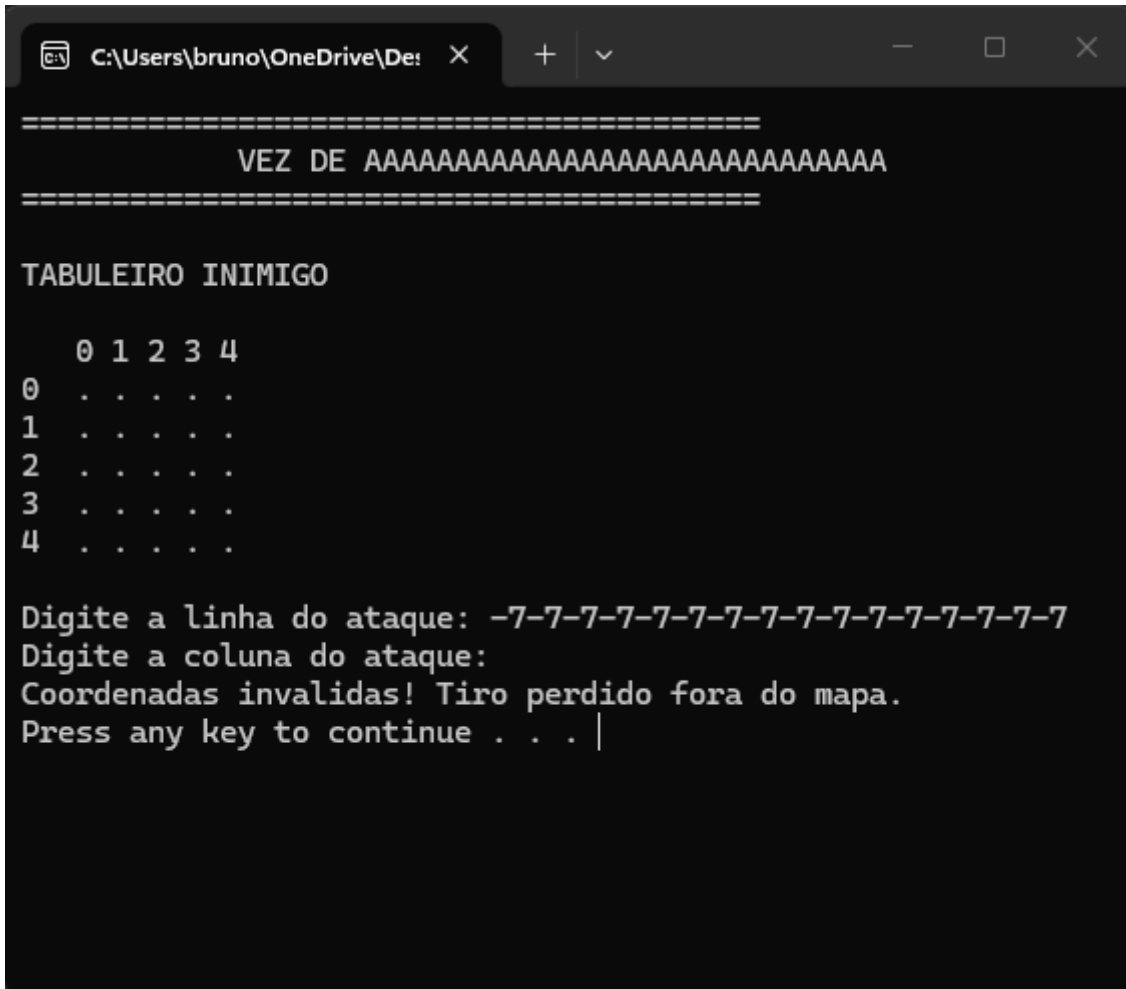
Uso estratégico do comando `break` dentro do laço `for` de verificação de acerto. Assim que o sistema localiza qual dos 3 navios inimigos foi atingido, ele encerra a busca imediatamente para poupar processamento, computa o acerto e força a alternância justa de turnos.

```
244  /* =====
245  MÓDULO 6: INTERRUPTÃO DE LAÇO E PASSAGEM ALTERNADA DE TURNO (BREAK)
246  - Por que é estratégico: Assim que o sistema localiza qual dos 3 navios
247  | foi atingido, ele encerra a busca imediatamente via 'break' para poupar
248  | processamento, computa o acerto e força a alternância justa de turnos.
249  =====
250  */
251  for(i = 0; i < 3; i++) {
252      if(linha == navio2Linha[i] && colunas == navio2Coluna[i]) {
253          if(tabuleiro2[linha][colunas] != 'X') {
254              printf("\nBOOOOM! NAVIO ATINGIDO!\n");
255              tabuleiro2[linha][colunas] = 'X';
256              acertos1++;
257          } else {
258              printf("\nAtencao: Voce ja tinha acertado este navio antes!\n");
259          }
260          acertouNavio = 1;
261          break; // Interrompe o loop de busca na mesma hora
262      }
263  }
```

2.4. Testes do Sistema (Jogo)

Durante as sessões de homologação, o grupo realizou testes de estresse focados em quebrar a robustez lógica do programa:

- Teste de Entrada de Texto Inválida (Injeção de Caracteres): Digitou-se a sequência -7-7-7-7-7-7-7 nas coordenadas de ataque. O sistema interceptou a falha do buffer, exibiu o aviso de erro e manteve o fluxo ativo sem sofrer travamento.



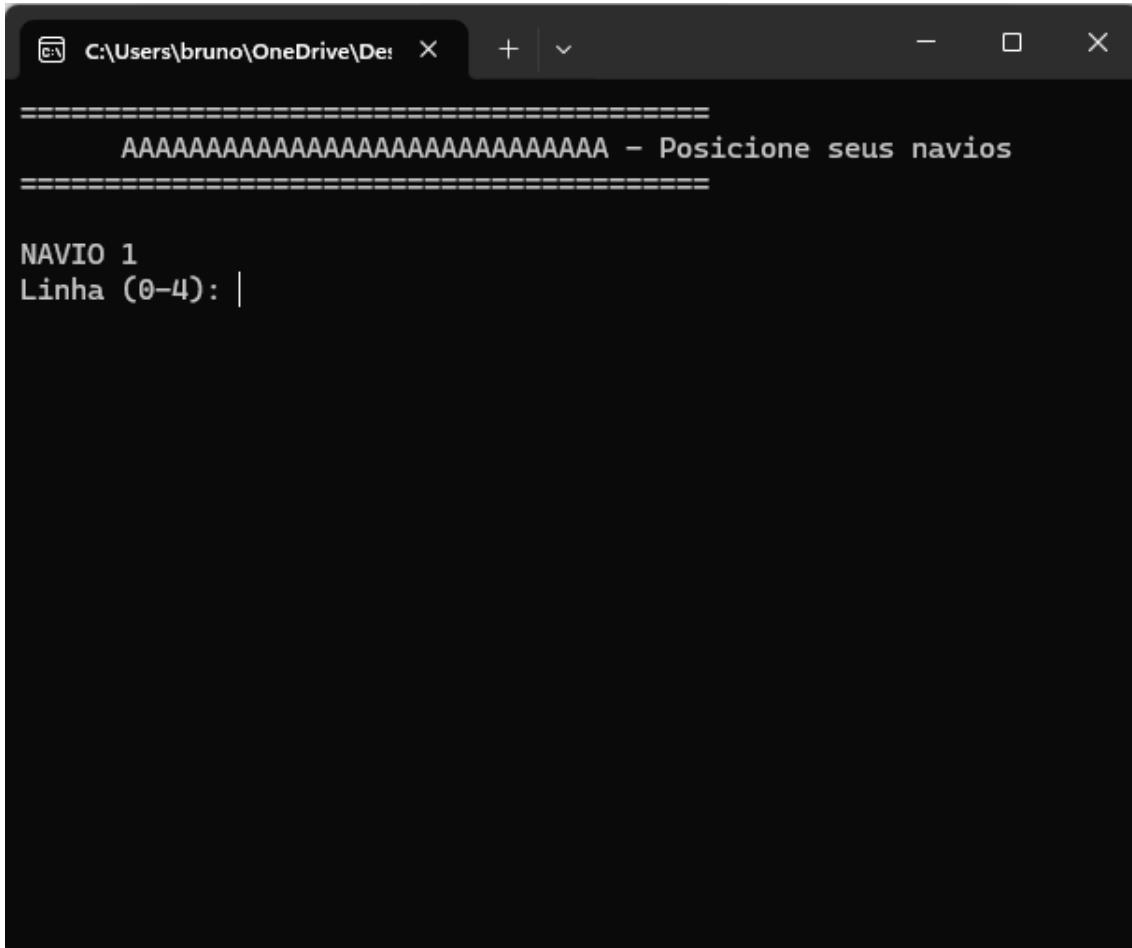
```
=====  
VEZ DE AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
=====
```

TABULEIRO INIMIGO

	0	1	2	3	4
0	.	.	.	.	.
1	.	.	.	.	.
2	.	.	.	.	.
3	.	.	.	.	.
4	.	.	.	.	.

Digite a linha do ataque: -7-7-7-7-7-7-7-7-7-7-7-7-7-7-7-7-7
Digite a coluna do ataque:
Coordenadas invalidas! Tiro perdido fora do mapa.
Press any key to continue . . . |

- Teste de Nomes Longos com String Limiar: Digitação de uma string gigante no campo de nomes. O sistema limitou a captura até as 29 primeiras letras, eliminou o excesso e manteve o leiaute alinhado.



```
C:\Users\bruno\OneDrive\De: X + v - □ X
=====
AAAAAAAAAAAAAAAAAAAAAAAAAAAA - Posicione seus navios
=====

NAVIO 1
Linha (0-4): |
```

- Teste de Tentativa de Sobreposição: Inserção intencional de 2 navios na mesma casa [0][0]. O sistema bloqueou o cadastro imediatamente a partir do segundo navio, gerando o alerta visual.

```
C:\Users\bruno\OneDrive\De: X + v - □ X
=====
AAAAAAAAAAAAAAAAAAAAAAAAAAAA - Posicione seus navios
=====

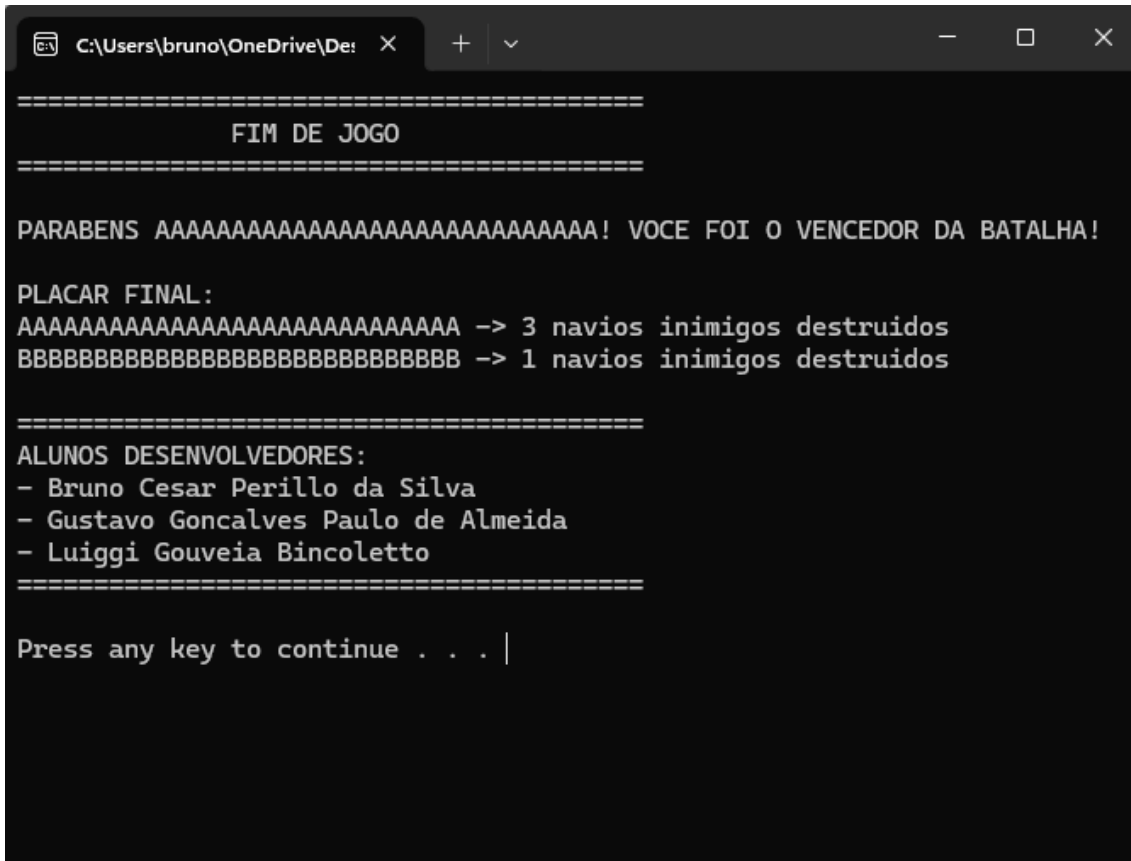
NAVIO 1
Linha (0-4): 0
Coluna (0-4): 0

NAVIO 2
Linha (0-4): 0
Coluna (0-4): 0

[ERRO] Voce ja colocou um navio nesta posicao! Escolha outra.

NAVIO 2
Linha (0-4): |
```

- Teste de Condição Terminal no Executável (.exe): Partida simulada executada direto pelo arquivo .exe. O jogo interrompeu os turnos no momento do terceiro acerto, exibiu o placar final com os créditos e manteve o terminal retido em tela.



```
=====
                        FIM DE JOGO
=====

PARABENS AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA! VOCE FOI O VENCEDOR DA BATALHA!

PLACAR FINAL:
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA -> 3 navios inimigos destruidos
BBBBBBBBBBBBBBBBBBBBBBBBBBBBBB -> 1 navios inimigos destruidos

=====
ALUNOS DESENVOLVEDORES:
- Bruno Cesar Perillo da Silva
- Gustavo Goncalves Paulo de Almeida
- Luigi Gouveia Bincoletto
=====

Press any key to continue . . . |
```

2.5. Manual de Uso do Sistema (Jogo)

Siga as instruções abaixo para a execução e compilação do software:

Como Compilar o Código: Abra o Prompt de Comando (CMD) do Windows, navegue até o diretório do projeto utilizando o comando:

```
| cd C:\Users\bruno\OneDrive\Desktop\C99\
```

Em seguida, execute a diretiva do compilador GCC:

```
| gcc batalha_naval.c -o jogo.exe
```

Como Executar no Terminal: No terminal do console, execute o arquivo binário gerado para iniciar o programa:

```
| jogo.exe
```

Como o Usuário Faz as Jogadas: Na fase de configuração, insira os nomes dos jogadores. Em seguida, digite o número da linha (0 a 4) seguido da coluna (0 a 4) para cada um dos 3 navios. Na fase de combate, digite alternadamente as coordenadas do quadrante inimigo que deseja alvejar.

Como Reiniciar ou Encerrar o Jogo: O encerramento ocorre de maneira autônoma quando um jogador acumular 3 pontos de impacto. Para fechar o programa a qualquer instante da execução, utilize a interrupção física de teclado Ctrl + C no console.

2.6. Demonstração em Vídeo e Recursos Digitais

Como complemento material e comprovação funcional da engenharia de blindagem adotada, o grupo gravou uma apresentação técnica em vídeo detalhando o código-fonte em tempo de execução e realizando testes de estresse em tempo real no console de comandos.

O material audiovisual foi disponibilizado publicamente na plataforma YouTube para livre avaliação. Caso ocorram limitações de hiperlink diretamente pelo corpo deste documento PDF, o link de acesso direto encontra-se transcrito textualmente logo abaixo:

Link de Acesso Direto (YouTube): <https://youtu.be/-aY1gQ8QD-k>

3. RESULTADOS E DISCUSSÃO

O simulador de Batalha Naval funcionou em conformidade com as metas estipuladas. A maior facilidade do projeto centrou-se na renderização visual das matrizes na tela em formato de grade, visto que a linguagem C manipula de forma ágil laços do tipo for encadeados para exibição de texto.

Por outro lado, o maior desafio técnico enfrentado foi lidar com a instabilidade crônica de buffer do console. Nas versões iniciais, resíduos de formatação deixados pelo scanf geram o "Efeito Fantasma", fazendo com que mensagens importantes de erro fossem limpas da tela por comandos system("cls") subsequentes sem que os usuários conseguissem ler, pulando os turnos de combate de forma inadequada. Além disso, nomes gigantes causavam estouro de buffer de string (String Buffer Overflow), corrompendo as variáveis próximas da pilha de memória.

A solução encontrada para estabilizar o software foi reestruturar todas as validações através da leitura do retorno numérico do scanf aliado à máscara rígida de comprimento %29s nas strings, limpando manualmente o excesso do buffer caractere por caractere via laço while(getchar() != '\n'). Essa decisão técnica eliminou os bugs de travamento por injeção de dados inadequados e garantiu fluidez nas partidas.

4. CONSIDERAÇÕES FINAIS

O projeto atingiu com êxito os objetivos acadêmicos propostos, proporcionando aos desenvolvedores um profundo entendimento prático acerca de gerenciamento de buffers de entrada e consistência de tipos primitivos em programação estruturada. A experiência provou que o desenvolvimento de softwares estáveis exige técnicas rigorosas de filtragem de dados antes de sua gravação definitiva nas variáveis principais.

Como sugestões de melhorias técnicas para versões futuras, o grupo aponta a necessidade de expansão do tamanho da matriz oceânica para 10×10 e a implementação de tamanhos variados de barcos (como fragatas e contratorpedeiros). Adicionalmente, planeja-se a codificação de um algoritmo de Inteligência Artificial básica baseado em números pseudo aleatórios para viabilizar partidas em modo singleplayer contra o computador, acoplado a um banco de dados estático para armazenamento de recordes e placares históricos.

REFERÊNCIAS

DEITEL, Paul; DEITEL, Harvey. C: como programar. 6. ed. São Paulo: Pearson, 2011.

SCHILD, Herbert. C: completo e total. 3. ed. São Paulo: Makron Books, 1997.